HPC School Bangkok
ZERO
5-7 August 2026
Faculty of Science, Kasetsart University
Bangkok Thailand

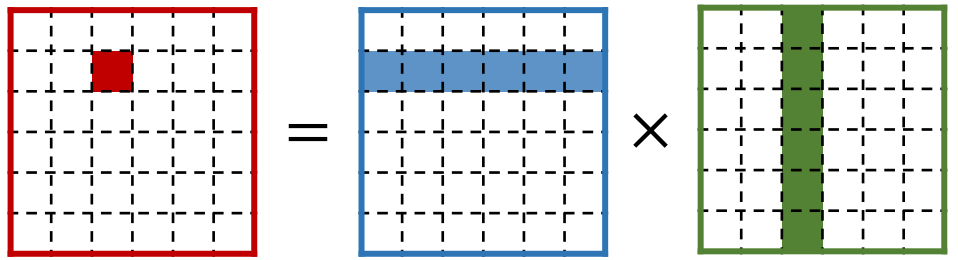
HPC Performance Profiling & Tuning for Non-Specialists

--- Workshop Session ---

Somrath Kanoksirirath
Scientific Support and Domain Research (SSD)
ThaiSC A National Science and Technology Infrastructure of NSTDA

Outline

Study case: Matrix multiplication



$C_{1,2}$ $A_{1,k}$ $B_{k,2}$

Using HPE Cray Perftools (CrayPAT)

perftools-preload	perftools	perftools-lite-xxx

`#!/bin/bash`
`#SBATCH ...`
 	
`ml ...`
`ml` **`perftools-base`**
`ml` **`perftools-preload`**
 	
`export PAT_RT_PERFCTR=…`
 	
`srun` **`pat_run [opts]`** `./matmul.exe`
 	
`pat_report [opts]` `\`
`./<Program>+<PID>-<Node><s,t>`
	`#!/bin/bash`
	`#SBATCH ...`
	
	`ml ...`
	`ml` **`perftools-base`**
	`ml` **`perftools`**
	
	`CC -o ./matmul.exe`
	`./matmul_main.cpp`
	
	`pat_build [opts]` `./matmul.exe`
	
	`export PAT_RT_PERFCTR=…`
	
	`srun ./matmul.exe+pat`
	
	`pat_report [opts]` `\`
	`./<Program>+<PID>-<Node><s,t>`
		`#!/bin/bash`
		`#SBATCH ...`
		
		`ml ...`
		`ml` **`perftools-base`**
		`ml` **`perftools-lite-xxx`**
		
		`CC -o ./matmul.exe`
		`./matmul_main.cpp`
		
		`export PAT_RT_PERFCTR=…`
		
		`srun ./matmul.exe`
		
		`#pat_report ... # <- Optional`

Using CrayPAT with Basic PAPI Counters

perftools-preload	perftools	perftools-lite-xxx
`#!/bin/bash`	`#!/bin/bash`	`#!/bin/bash`
`#SBATCH ...`		
`ml ...`		
`ml perftools`		
`ml perftools`		
`export PAT_RT_PERFCTR=...`	**`export PAT_RT_PERFCTR=...`**	**`export PAT_RT_PERFCTR=...`**
`srun pat_run [opts] ./matmul.exe`	`srun ./matmul.exe+pat`	`srun ./matmul.exe`
`pat_report [opts] \` `  ./<Program>+<PID>-<Node><s,t>`	`pat_report [opts] \` `  ./<Program>+<PID>-<Node><s,t>`	`#pat_report ... # <- Optional`

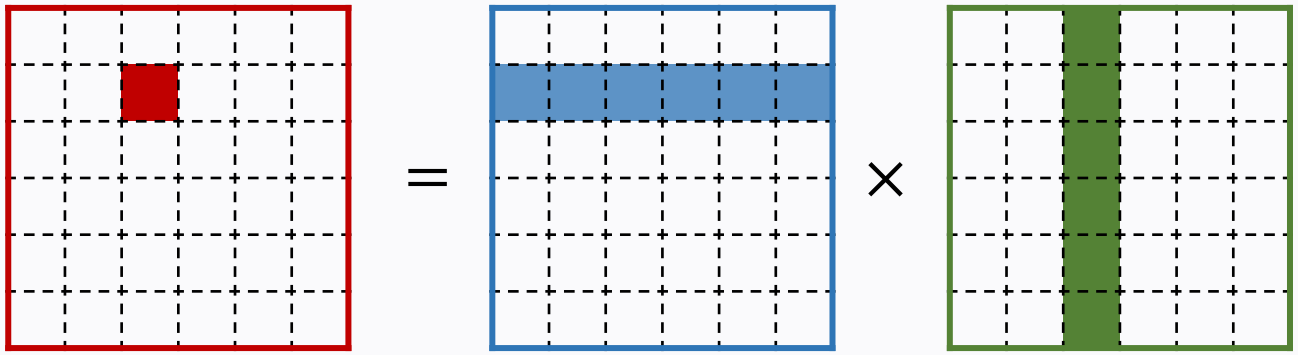
```
# Instruction
export PAT_RT_PERFCTR="PAPI_TOT_CYC,PAPI_TOT_INS,PAPI_VEC_INS"
export PAT_RT_PERFCTR="${PAT_RT_PERFCTR},PAPI_FP_INS,PAPI_FP_OPS"
export PAT_RT_PERFCTR="${PAT_RT_PERFCTR},PAPI_BR_INS,PAPI_BR_MSP"

# Cache
export PAT_RT_PERFCTR="PAPI_L1_DCA,PAPI_L1_DCM,PAPI_L2_DCM"
export PAT_RT_PERFCTR="${PAT_RT_PERFCTR},PAPI_TOT_INS,PAPI_TLB_DM"
```

Study case: Matrix multiplication (matmul)

Matrix multiplication is a simple linear algebra operation but plays a pivotal role in computer simulations, particularly in CFD. For example, a simple 1D backward Euler scheme for solving an advection problem

$$\frac{u_i^{n+1}-u_i^n}{\Delta t} + C\,\frac{u_{i+1}^{n+1}-u_i^{n+1}}{\Delta x} = 0$$

is, in fact, an $AX_{n+1} = X_n$ problem where **num rows = num cols = the total number of grid points**!!
After solving for the inverse, it is multiplied iteratively to advance the simulation: $X_{n+1} = A^{-1}X_n$



$C_{1,2}$ $A_{1,k}$ $B_{k,2}$

Trivial code: matrix multiplication

```
// Dimension [MxP] = [MxN] x [NxP]

for(int i=0; i<M ; i+=1)
for(int j=0; j<P ; j+=1)
{
    Real temp = 0 ;
    for(int k=0; k<N ; k+=1)
    {
        temp += A[N*i+k] * B[P*k+j] ;
    }
    C[P*i+j] = temp ;
}
```

Matmul program -- Pseudo code

Imitated program flow: matmul_main.cpp

```
Get parameters
Print parameters
Allocate A,B,C arrays
Main loop [NUM_REPEAT]:
    Initialize A,B with random numbers
    Initialize C with zeros
    Do matrix multiplication ---
    If [WRITE_ARRAYS]
        write outputs
Deallocate A,B,C arrays
```

Trivial code: matrix multiplication

```
// Dimension [MxP] = [MxN] x [NxP]

for(int i=0; i<M ; i+=1)
for(int j=0; j<P ; j+=1)
{
    Real temp = 0 ;
    for(int k=0; k<N ; k+=1)
    {
        temp += A[N*i+k] * B[P*k+j] ;
    }
    C[P*i+j] = temp ;
}
```

- Outer [1] -- REPEAT loop (Iteration)
- Middle [2] -- Loop over C matrix
- Inner [1] -- Loop along N dimension

Matmul program -- Directory tree

```
├── README
├── basic/          ---   Self-study
├── single/
│   ├── papi/       ---   Lab 1
│   └── tune/       ---   Lab 2 & 3
├── parallel/
│   ├── omp/        ---   Lab 4
│   ├── mpi/        ---   Lab 5
│   └── gpu/        ---   Lab 6
└── src/            ---   Source code
```

```
└── src/
    ├── matmul_algor.hpp    ---   USE_ALGOR=0-4  (+OMP)
    ├── matmul_kernel.hpp   ---   micro_kernel() for USE_ALGOR=3
    ├── matmul_main.cpp     ---   main loop  (+MPI)
    ├── matmul_setup.hpp    ---   parameters and directives ***
    ├── matmul_util.hpp     ---   random, partition_dim, write_*,
    │                             clock/timer, ...
    ├── check.py            ---   check the result,
    │                             -DWRITE_ARRAYS=1 --> C_[iter].bin
    └── matmul.py           ---   a reference using NumPy
```

Matmul program -- Usage

- **matmul_setup.hpp**

  = contains default input arguments, compile-time constants, program directives, ...,

  e.g., DEFAULT_MATRIX_X_SIZE, BLOCK_X_SIZE, MICRO_X_SIZE, ...

- **Compile-time macros/directives**

  = passing -DXXX=[N] at compile time will override the default directives defined in matmul_setup.hpp,

  e.g., -DUSE_ALGOR=[N], -DUSE_OMP=[N], -DUSE_MPI=[N], -DUSE_MPI_IO=[N]

- **Runtime input arguments**

  = srun ./matmul.exe [niter] [M] [N] [P] where C[M, P] = A[M, N] x B[N, P]

- **Standard output**

  1) Program information, matrix sizes, algorithm employed, and others
  2) Current iteration, showing progress
  3) Two timers displayed at the end: Main loops. and Whole program.

- **Checking the file outputs, i.e., A_[niter].bin, B_[niter].bin, C_[niter].bin**

  = submit a job running “python3 check.py [niter] [M] [N] [P] [IO_step]” --> see …./basic/submitPyCheck.sh

LAB 1 :: PAPI Performance Counter

(~30-40 min)

Matmul Programs in LAB 1

Python-Numpy

./matmul.py

```
import numpy as np

C = np.matmul(A, B)
```

Trivial code

-DUSE_ALGOR=0 ./matmul_main.cpp

```
for(int i=0; i<M ; i+=1)
for(int j=0; j<P ; j+=1)
{
    Real temp = 0 ;
    for(int k=0; k<N ; k+=1)
    {
        temp += A[N*i+k] * B[P*k+j] ;
    }
    C[P*i+j] = temp ;
}
```

Loop interchange

-DUSE_ALGOR=1 ./mamul_main.cpp

```
for(int i=0; i<M ; i+=1)
for(int k=0; k<N ; k+=1)
{
   for(int j=0; j<P ; j+=1)
   {
      C[P*i+j] += A[N*i+k] * B[P*k+j] ;
   }
}
```

P1 | LAB 1 :: PAPI Performance Counter

First, copy the matmul directory to your working path.
It will be your working directory for the entire workshop.

1. Duplicate the **.../matmul/single/papi/** directory for each experiment below, to avoid conflicts when running them at the same time.

   a) Use the **submitPreload.sh** to profile the matmul.py program

   b) Use the **submitPerf.sh** to profile the matmul.exe program built with **-DUSE_ALGOR=0** and **-O3**

   c) Edit the **submitPerf.sh** to profile the matmul.exe program built with **-DUSE_ALGOR=1** and **-O3**. Then, run it

Then, calculate some of the metrics learned in the lecture, identify major bottlenecks, then compare and discuss.

[Tips: Use “diff -y --width=[N] [report_file1] [report_file2]” and/or “grep ‘Whole’ [log_files]” to view the results]

P2 | LAB 1 :: PAPI Performance Counter

2. Duplicate the **.../matmul/single/papi/** directory to avoid conflicts.

   Use **submitPerf_Array.sh** to run the matmul program built with **-DUSE_ALGOR=0** while varying the matrix dimensions,

   using Slurm job arrays, **from M=N=P=250 to 260** and **from M=N=P=510 to 520**,

   (so 2x10 = 20 runs)

   - Use “grep ‘Whole’ slurm*.out”. What can be observed from the differences in their runtimes? Does any run stand out?
   - What are the differences in their PAT reports?
   - What is likely the cause of the behavior observed? What is it called? What dimension plays the most important role?
   - Will we observe the same behavior if -DUSE_ALGOR=1 is used instead?
   - [Extra] Can you explain the mechanism numerically?
   - [Extra] What is the critical value that could cause huge L1, L2 cache misses? <-- DO NOT run it!

LAB 2 :: Cache Optimization Techniques

and

LAB 3 :: Loop Unroll & SIMD Vectorization

(Totally ~50-60 min)

Matmul Programs in LAB 2 & 3

Cache blocking

-DUSE_ALGOR=2 ./matmul_main.cpp

```
for(int jj=0 ; jj < p ; jj+=BLOCK_P_SIZE)
for(int kk=0 ; kk < n ; kk+=BLOCK_N_SIZE)
{
  // <-- Pack B here
  for(int ii=0 ; ii < m ; ii+=BLOCK_M_SIZE)
  {
    // <-- Pack A here
    for(int i=ii ; i < ii+BLOCK_M_SIZE ; ++i)
    for(int k=kk ; k < kk+BLOCK_N_SIZE ; ++k)
    {
      for(int j=jj ; j < jj+BLOCK_P_SIZE ; ++j)
        CC[p*i+j] += AA[n*i+k] * BB[p*k+j] ;
    }
  }
}
```

Cache blocking + Micro-kernel

-DUSE_ALGOR=3 ./matmul_main.cpp

```
void micro_kernel<Mc,Pc>(...)
{
  cc[Mc*Pc] = {0} ;
  for(int k=0 ; k<num_k ; ++k)
  {
    for(int i=0 ; i<Mc ; ++i)
    for(int j=0 ; j<Pc ; ++j)
     cc[i,j] += A[i,k] * B[k,j];
  }
  for(int i=0 ; i<Mc ; ++i)
  for(int j=0 ; j<Pc ; ++j)
    C[i,j] += cc[i,j];
}
```

Cblas routine

-DUSE_ALGOR=4 -DSELECT_BLAS=1 ./matmul_main.cpp

```
#include <mkl.h>

cblas_dgemm(CblasRowMajor, CblasNoTrans, CblasNoTrans, \
            ...
```

General instructions for LAB 2 & 3

1) Duplicate the **.../matmul/single/tune/ under** the same path to run several matmul jobs in parallel and avoid conflicts***
2) Only make changes to the parameters in the local header files, i.e., matmul_setup.hpp, matmul_kernel.hpp
3) Number of repeats (or iterations) = 20
4) Matrix dimension: M ≥ 1000, N ≥ 1000, P ≥ 1000
5) Every matrix dimension must be divisible by its respective block size, e.g., MATRIX_M_SIZE % BLOCK_M_SIZE == 0
6) Every block size must be divisible by its respective micro-kernel size, e.g., BLOCK_M_SIZE % MICRO_M_SIZE == 0
7) Use only one single CPU core
8) Use the Intel compiler and -O3 optimization flag, unless stated otherwise
9) Use double-precision floating-point data type (64-bit, 8-byte), unless stated otherwise
10) Without file I/O, that is, -DWRITE_ARRAYS=0 or no setting, unless stated otherwise

Note: 5 and 6 are required because edge cases are not handled--we prefer code readability.

P3 | LAB 2 :: Cache Optimization Techniques

3. You can do (a), (b) and (c) in any order, while leaving (d) for last.
   a) Use **submitRef.sh** to run the MKL dgemm reference program, obtaining its runtime and profiling report as a baseline.
   b) Using **submitSingle.sh**, tune the matmul program built with **-DUSE_ALGOR=2** by adjusting the **block size** parameters in the local matmul_setup.hpp, as well as the matrix dimensions.
   c) Using **submitSingle.sh**, tune the matmul program built with **-DUSE_ALGOR=3** by adjusting the **micro size** parameters in the local matmul_setup.hpp, using your final block sizes from (3.b).
   d) Use **submitFinal.sh** to run your final program and the reference programs without profiling. Change the input arguments in the job script as you want.

Note1: You need to redo Step (3.a) if you change the matrix dimensions in Step (3.b) or (3.c).

Note2: Your final program should run **no more than 10% slower than the reference**.

Hint1: The block may not need to be entirely in the L1, but should be in L2? What is the order of magnitude?

Hint2: What micro sizes are small enough to fit in the registers and can fully utilize AVX2 as well?

P4 | LAB 3 :: Loop Unroll & SIMD Vectorization

4. Using your matmul_setup.hpp from Problem 3 (LAB 2), edit **submitSingle.sh** to build your final tuned matmul program (-DUSE_ALGOR=3) with -qopt-report=3 -qopt-report-phase=vec,loop and, one at a time, with

   I. **-DALGOR3_NO_MANUAL_VEC_KERNEL=0 -qopt-report-file=./optrpt_manual_vec.txt**

   II. **-DALGOR3_NO_MANUAL_VEC_KERNEL=1 -qopt-report-file=./optrpt_auto_vec.txt**

   Then,

   - Compare the two runs, including their runtimes and the generated reports.
   - Tune the slower program to achieve the same performance by adding compiler flags and/or compiler optimization directives, such as **PRAGMA_UNROLL(N), PRAGMA_VECTORIZE, ...** to the loops inside the **micro_kernel_scalar()** function--defined in the local matmul_setup.hpp and the local matmul_kernel.hpp, respectively.
   - Reflect on the two approaches taken to optimize the program. If you were a developer, which would you choose?

P5, P6 | LAB 3 :: Loop Unroll & SIMD Vectorization

5. [Extra] Verify the correctness of both approaches in Problem 4 by adding

   -DWRITE_ARRAYS=1 -DNUM_ITER_PER_WRITE=[IO_Step]

   then running "python3 ./check.py [niter] [M] [N] [P] [IO_Step]", using .../matmul/basic/submitPyCheck.sh as a template. Vary the problem size and other parameters as you want.

6. [Extra] Based on your experience and understanding developed throughout the workshop, tune the single-precision floating-point version of the matmul program, by setting **"using Float = float;"** in the local **matmul_setup.hpp**.

   After that, answer the following questions:

   - Compare the two floating-point versions: runtime, profiling report, and Intel optimization report (as in Problem 4)
   - Check the correctness of the results as in Problem 5
   - Discuss the pros and cons of the two floating-point versions, especially on the points learned in the lecture.

   Note: You should be able to guess proper BLOCK sizes and MICRO sizes without haphazardly varying them.

LAB 4 :: Shared Memory Parallelism

(~35-45 min)

Parallelized Matmul Program using OpenMP

USE_OMP =
1 (no collapse),
2 (with collapse)

- [USE_ALGOR=0, 1, 2, 3] The outermost loop is parallelized using

  "#pragma omp parallel for [collapse (n)] … schedule(runtime)"

- [USE_ALGOR=4] Multi-threading is handled by the CBLAS implementation, e.g., -qmkl=parallel

-DUSE_ALGOR=0 (Trivial code)

```
#pragma omp parallel for ...
for(int i=0; i<M ; i+=1)
for(int j=0; j<P ; j+=1)
{
  Real temp = 0 ;
  for(int k=0; k<N ; k+=1)
  {
    temp += A[N*i+k]*B[P*k+j];
  }
  C[P*i+j] = temp ;
}
```

-DUSE_ALGOR=1 (Loop interchange)

```
#pragma omp parallel for ...
for(int i=0; i<M ; i+=1)
for(int k=0; k<N ; k+=1)
{
  for(int j=0; j<P ; j+=1)
  {
    Float temp = A[N*i+k]*B[P*k+j];
    #pragma omp atomic // if collapse
    C[P*i+j] += temp ;
  }
}
```

-DUSE_ALGOR=2,3 (Cache blocking)

```
#pragma omp parallel for ...
for(...; jj < p ; jj+=BLOCK_P_SIZE)
for(...; ii < m ; ii+=BLOCK_M_SIZE)
for(...; kk < n ; kk+=BLOCK_N_SIZE)
{
  for(...; i < ii+BLOCK_M_SIZE ; ++i)
  for(...; k < kk+BLOCK_N_SIZE ; ++k)
  for(...; j < jj+BLOCK_P_SIZE ; ++j)
  {
    CC[p*i+j] += AA[n*i+k]*BB[p*k+j];
  }
}
```

P7 | LAB 4 :: Shared Memory Parallelism

Copy your final local matmul_xxx.hpp from LAB 3 to replace the global header files in your **.../matmul/src/**

In this lab, duplicate the **.../matmul/parallel/omp/** directory under the same path at the beginning of each run

7. Execute "**sbatch submit_atomic.sh**" to submit two jobs (job array), using 8 threads, running the program built with **-DUSE_ALGOR=1 -DUSE_OMP=1 or 2**---there should be no need to edit the script.

   Then, based on the tracing reports in ./omp1/ and ./omp2/ directories,

   - Discuss their runtimes, OpenMP overhead, synchronization, load balance, ..., cache access, ...
   - From the code in the previous slide, what is the cause of this huge performance difference?
   - What did you learn from this lab? Especially, if you were a developer.

P8 | LAB 4 :: Shared Memory Parallelism

8. Execute “**sbatch submit_outer.sh**” to submit several jobs (job array), using 64 threads, running the program built with **-DUSE_ALGOR=0 -DUSE_OMP=2** but with **different OMP_SCHEDULE chunk size**.

   Remove the “chunk_[N]” directories and rerun (1-2 times), keeping all the slurm-*.out files.

   Then,

   - Execute “grep ‘Whole’ slurm-*” to inspect the runtime of every job at the same time. Plot it (optional).
   - Discuss the trend and explain its cause, technically (and numerically).
   - Set **enable_perftools_trace=1** in the job script then rerun to generate and compare profiling reports.

     Can this performance drawback get detected by the tool? Where is it indicated in the report?

   Hint: Study the script, then write out the memory access pattern of each thread

P9 | LAB 4 :: Shared Memory Parallelism

9. In this task, you will execute "**bash auto_omp_job.sh**" (not sbatch!) which will send jobs (one per second) to Slurm.

   First, set the **jobs_input_args='[niter] [M] [N] [P]'** in the script where niter = 50 and 3000 ≤ M, N, P ≤ 5000.
   As before, they must also satisfy the conditions for USE_ALGOR=3 mentioned during LAB 2 and 3.

   Then, send several jobs, one per directory

   I. With **jobs_num_thread=("64" "32" "16" "8" "4" "2" "1")** and **jobs_num_task=("1" "2" "4" "8" "16" "32" "64")** to concurrently utilize all cores in a socket, limiting CPU frequency boost to some extent.

   II. With **jobs_num_thread=("64" "32" "16" "8" "4" "2" "1")** but **jobs_num_task=("1" "1" "1" "1" "1" "1" "1")**

   Then,

   - Use "grep 'Whole' ./c*times/slurm-*_0.out" to inspect runtime of every job at the same time.
   - Calculate the speedup and discuss the scalability of the program in both situations. Any comments?
   - [Extra] In the script, set **enable_perftools_trace=1** then run **jobs_num_thread=("4") jobs_num_task=("16")** and **jobs_num_thread=("4") jobs_num_task=("1").** What are the differences in the generated reports? Discuss.
   - [Extra] Do you think --cores-per-socket=64 is important? What does it do?

LAB 5 :: Distributed Memory Parallelism

(~40-60 min)

Parallelized Matmul Program using MPI

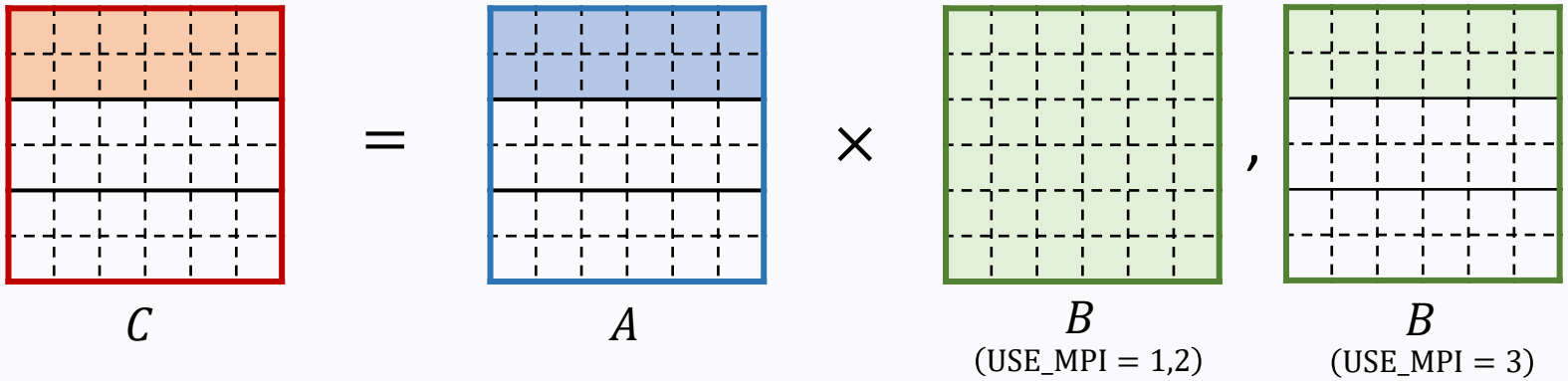
USE_MPI =

1 (MPI_Bcast),

2 (MPI_Ibcast),

3 (MPI_Iallgatherv)

- The workload (matrix C) is divided along the M dimension.
- The corresponding part of matrix A is locally randomized by each MPI process.
- [USE_MPI=1,2] The whole matrix B is randomized by rank 0, then broadcast to the other ranks
  - ➢ Set **NUM_SPLIT_MPI_CALL** to break the message into smaller parts.
- [USE_MPI=3] Matrix B is subdivided into parts as matrices A and C. Each part is randomized by a rank. Then, every rank gathers all the parts to get the entire matrix B for subsequent calculation.



Parallelized Matmul Program using MPI

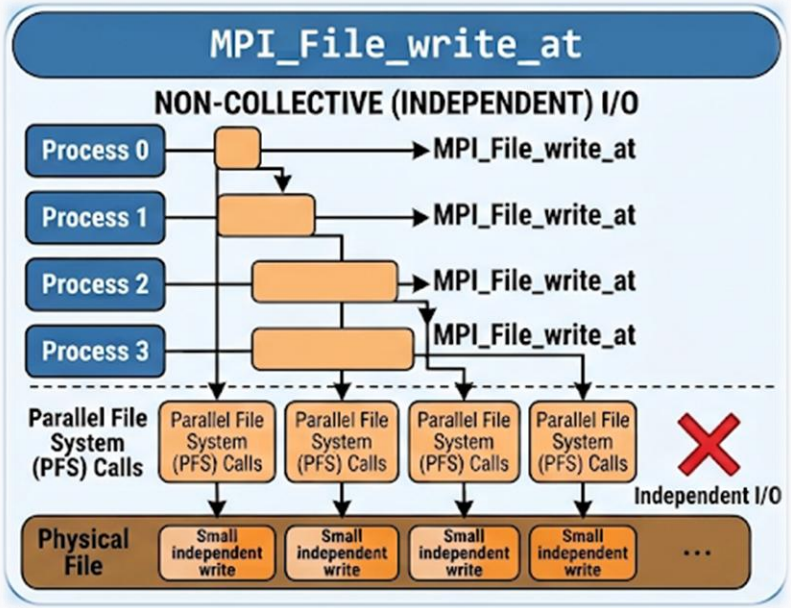
USE_MPI_IO =

0 (Gather -> Serial),

1 (MPI_File_write_at),

2 (MPI_File_write_at_all)

- [USE_MPI_IO=0] Matrices A and C are gathered at rank 0, then all matrices are written out.
- [USE_MPI_IO=1] Every MPI rank writes a portion of matrices A, B, C to the output files individually.
- [USE_MPI_IO=2] Every MPI rank writes a portion of matrices A, B, C. The write requests can be coordinated and optimized by the MPI implementation through various I/O techniques.
  - MPICH_MPIIO_HINTS="<file_pattern0>[:hint01:hint02:...],<file_pattern2>[:hint11:...],..."



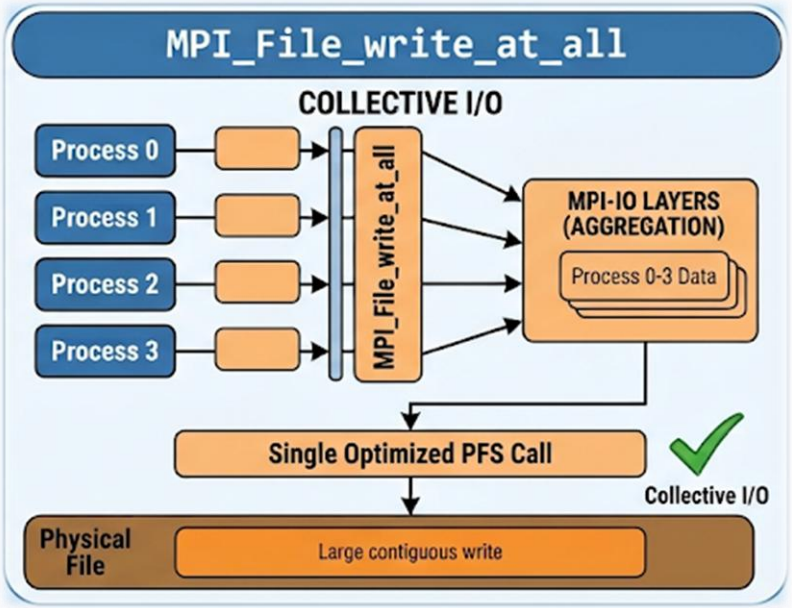


Image generated by Google Gemini.

P10 | LAB 5 :: Distributed Memory Parallelism

In this lab, duplicate the **.../matmul/parallel/mpi/** directory under the same path at the beginning of each run

10. In this task, you will execute "**bash auto_send_job.sh**" (not sbatch!) to send several jobs to Slurm, varying the number of MPI processes, i.e., **jobs_num_task**, over 1, 2, 4, 8, 16, 32, 64.

    First, set the **jobs_input_args='[niter] [M] [N] [P]'** in the script to the same values as in Problem 9, LAB 4.

    Next, one by one, edit and run the script using **USE_MPI=1, 2, and 3**---setting **jobs_prefix_dir** as you want.

    Then,

    - Apply the "grep" command to inspect the runtimes of the jobs. Compare and discuss the scalability of the three versions.
    - Investigate the tracing reports, focusing on MPI overhead, synchronization time, load (im)balance and others.
      Take note of the differences in the reports of the three versions and discuss.

P11 | LAB 5 :: Distributed Memory Parallelism

11. Adapt the same **auto_send_job.sh** in Problem 10 to find the optimal configuration of **-DUSE_MPI** and **-DUSE_OMP**, as well as **the number of OpenMP threads** and **the number of MPI processes** for

    I. The same problem size as in Problem 9 and 10, which satisfies niter = 50 and 3000 ≤ M, N, P ≤ 5000.

    II. A larger problem size that satisfies niter = 20 and 10,000 ≤ M, N, P ≤ 12,000

    Note1: We recommend setting enable_perftools_trace=0

    Note2: Do not use more than 256 CPU cores (or 2 reserved nodes) per job during the workshop.

    Note3: You may not need to test every possible combination for this problem. Understanding the idea is sufficient.

    Note4: If you have time, feel free to experiment with an odd number of MPI processes and/or OpenMP threads.

LAB 5 :: Distributed Memory Parallelism

12. Edit **submit_io.sh** to run the large problem size in Problem 11, where niter = 20 and 10,000 ≤ M, N, P ≤ 12,000, using the same optimal configuration you found.

    Ensure that **-DWRITE_ARRAYS=1** and **-DNUM_ITER_PER_WRITE=[N]** such that there will be **approximately 5 writes**.

    Submit the job script, which will vary **-DUSE_MPI_IO=0, 1, and 2** using a job array

    Then,

    - Apply the "grep" command to inspect runtime of the jobs. Discuss the result.
    - Try tuning **-DUSE_MPI_IO=2** using **MPICH_MPIIO_HINTS** to achieve better performance. Set --array=2 for this step.
    - Set **enable_perftools_trace=1** and rerun them all.
      Investigate the profiling reports, especially focusing on I/O time percentage, MPI-IO operations, file I/O statistics and Lustre information. Take note of the differences in the reports of the three versions and discuss.
    - [Extra] Investigate when tuned collective MPI-IO becomes significantly faster than non-collective MPI-IO?

    Note: You may look up information on MPI environment variables online or use "man intro_mpi".

LAB 6 :: GPU

[Extra, only if time permits]

P13 | LAB 6 :: GPU

For this lab, use the .../matmul/parallel/gpu/submitGPU.sh job script

13. Using a single GPU, build and run a matmul program with **-DUSE_ALGOR=4 -DSELECT_BLAS=2**

- Investigate the tracing reports, especially those that indicate GPU overhead and GPU computing time.
- Find one of the largest problem sizes requiring >10 GB VRAM (out of 40 GB of available VRAM).
- Compare them with the CPU cases. Any comments?

Note: You may monitor the job's GPUs by firstly execute "source /project/common/General/.extra" (once per login).

Then, use "gpu_usage <job-id>".

P14, P15 | LAB 6 :: GPU

14. Edit the job script to use 4 GPUs on a single GPU node. Then, one at a time, run a matmul program built with

    I. **-DUSE_MPI=2 -DUSE_MPI_GPU_DIRECT=0** and set **export MPICH_GPU_SUPPORT_ENABLED=0**

    II. **-DUSE_MPI=2 -DUSE_MPI_GPU_DIRECT=1** and set **export MPICH_GPU_SUPPORT_ENABLED=1**

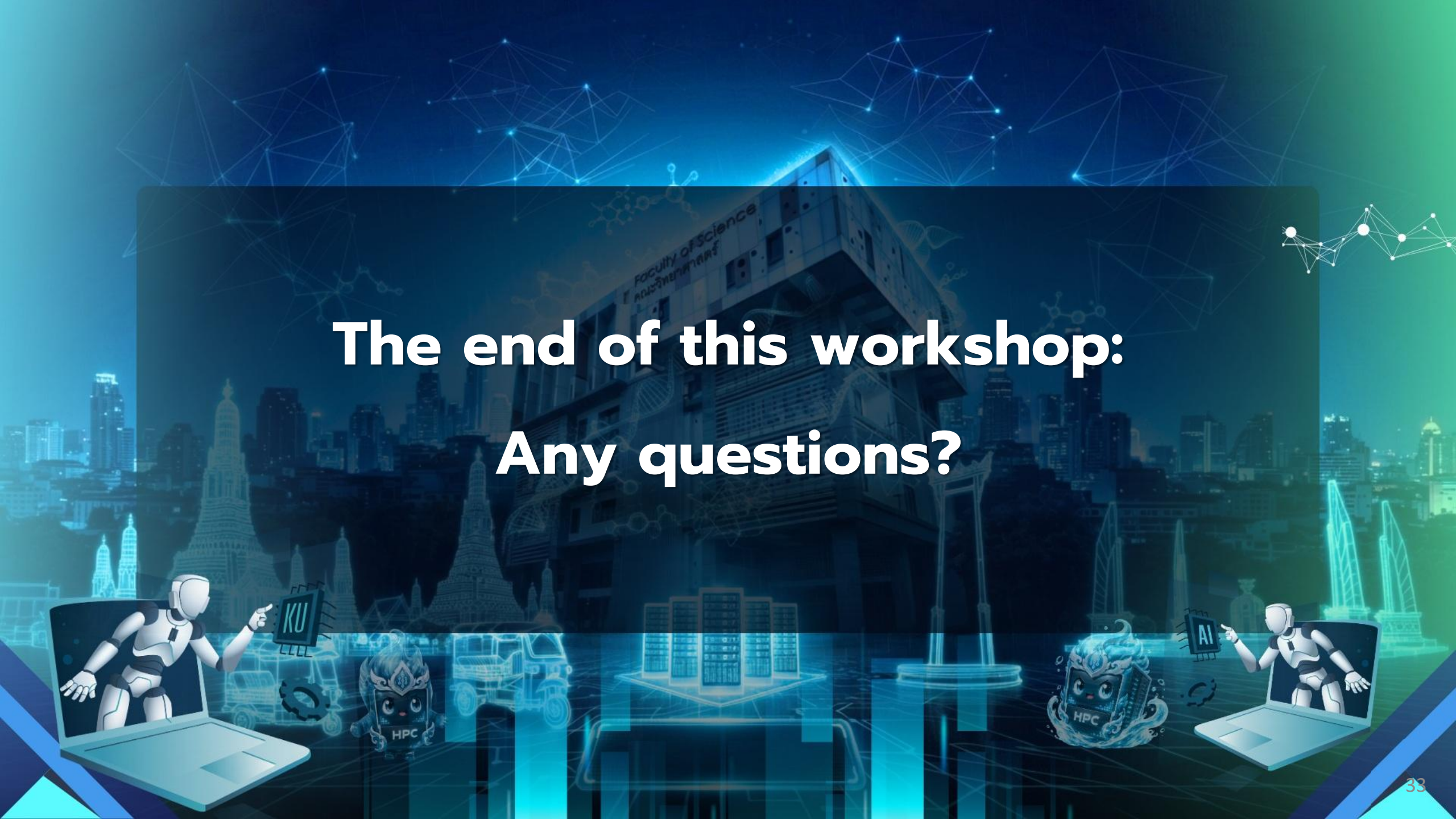
    Set #SBATCH --array=xxx to vary the matrix size from 10,000 to 30,000.

    Note: You may need to set -DNUM_SPLIT_MPI_CALL, if the Bcast message size is too large.

    Then,

    - Discuss the runtime
    - With GPUDirect (II), at what problem size does the GPU spends more than 50% of the runtime computing?

15. Repeat Problem 14 but use 4 GPUs on two different GPU nodes (2 GPUs per node), answering the same questions. After that, compare the results of Problem 14 (single-node) and Problem 15 (multiple nodes).



The end of this workshop:

Any questions?